FINDING TIME COMPLEXITY OF ALGORITHMS

Problem 1: Finding Complexity using Counter Method

```
#include<stdio.h>
 int main() {
    int n;
    scanf("%d",&n);
    int i=1;
 int s=1;
 int count=0;
 count++;
 count++;
 while(s<=n){
  count++;
 i++;
  count++;
  s+=i;
 count++;
  }
  count++;
 printf("%d",count);
 return 0;
}
```

Problem 2: Finding Complexity using Counter method

```
#include <stdio.h>
 int main() {
 int n, count = 0;
  scanf("%d", &n);
 if (n == 1) {
  count += 2;             }
```

```
 else {
  count++;
 for (int i = 1; i <= n; i++) {
 count += 5;
        }
   count++;
    }
    printf("%d", count);
  return 0;
}
```

Problem 3: Finding Complexity using Counter Method

```
#include<stdio.h>
 int main(){
  int num;
  scanf("%d",&num);
 int count=0;
 for(int i=1;i<=num;++i){
 count++;
 count++;
  if(num%i==0){
  count++;
        }
    }
     count++;
  printf("%d",count);
  return 0;
}
```

Problem 4: Finding Complexity using Counter Method

```
#include <stdio.h>
```

```
 int main() {
    int n;
    scanf("%d", &n);
  int count=0;
  count++;
 for (int i = n / 2; i < n; i++){
  count++;
  for (int j = 1; j < n; j =2*j){
  count++;
 for (int k = 1; k < n; k = k*2){
  count++;
 count++;
           }
  count++;
        }
        count++;
    }
    count++;

 printf("%d", count);
   return 0;
}
```

Problem 5: Finding Complexity using counter method

```
#include<stdio.h>
 int main(){
    int n;
    scanf("%d",&n);
    int rev=0;
```

```
   int remainder=0;
   int count=0;
  count++;
  count++;
  while(n!=0){
       count++;
    remainder=n%10;
    count++;
   rev=rev*10+remainder;
  count++;
 n/=10;
   count++;
    }
    count++;     printf("%d",count);
  return 0;
}
```

GREEDY ALGORITHMS

1-G-Coin Problem

```
#include <stdio.h
> int main() {
    int v;
  int count = 0;
  scanf("%d", &v);
  int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
   for (int i = 0; i < 9; i++) {
    count = count + v / coins[i];
      v = v % coins[i];
    }
```

```
    printf("%d", count);
  return 0;
}
```

2-G-Cookies Problem

```
#include <stdio.h>
 int main() {
    int n, m;
  scanf("%d", &n);
    int g[n];
   for (int i = 0; i < n; i++) {
   scanf("%d", &g[i]);
    }
    scanf("%d", &m);
    int s[m];
   for (int i = 0; i < m; i++) {
     scanf("%d", &s[i]);
    }
  for (int i = 0; i < n - 1; i++) {
  for (int j = i + 1; j < n; j++) {
    if (g[i] > g[j]) {
   int temp = g[i];
    g[i] = g[j];
  g[j] = temp;
           }
        }
    }
 for (int i = 0; i < m - 1; i++) {
     for (int j = i + 1; j < m; j++) {
     if (s[i] > s[j]) {
```

```
      int temp = s[i];
       s[i] = s[j];
      s[j] = temp;
            }
         }
    }
  int i = 0;
 int j = 0;
  int count = 0;
    while (i < n && j < m) {
        if (s[j] >= g[i]) {
       count++;
        i++;
            j++;         }
 else {
     j++;         }
    }
    printf("%d", count);
 return 0;
}
```

3-G-Burger Problem

```
#include <stdio.h>
 #include <math.h>
 int main()
{
   int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++)
```

```
    {
        scanf("%d", &arr[i]);
    }     for (int i = 0; i < n - 1; i++)
    {
    for (int j = 0; j < n - i - 1; j++)
        {
       if (arr[j] < arr[j + 1])
            {
                int temp = arr[j];
       arr[j] = arr[j + 1];
       arr[j + 1] = temp;
            }
        }
   }
    int s = 0;
     for (int i = 0; i < n; i++)
     {
  s += (int)(pow(n, i)) * arr[i];
     }
     printf("%d", s);
     return 0;
}
```

4-G-Array Sum max problem

```
#include <stdio.h>
 #include <stdlib.h>
 int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}
 int main() {
```

```
    int n;
    scanf("%d", &n);
    int arr[n];     for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
    }
    qsort(arr, n, sizeof(int), compare);     long long sum = 0;
    for (int i = 0; i < n; i++) {
    sum += (long long)arr[i] * i;
    }
    printf("%lld", sum);
   return 0;
}
```

5-G-Product of Array elements-Minimum

```
#include <stdio.h>
 int main() {
    int n, i, j, temp;
 long long sum = 0;
   scanf("%d", &n);
 int A[n], B[n];
   for (i = 0; i < n; i++)
   scanf("%d", &A[i]);
  for (i = 0; i < n; i++)
      scanf("%d", &B[i]);
   for (i = 0; i < n; i++) {
   for (j = i + 1; j < n; j++) {
    if (A[i] > A[j]) {
temp = A[i];
  A[i] = A[j];
   A[j] = temp;
```

```
            }
        }
    }
   for (i = 0; i < n; i++) {
     for (j = i + 1; j < n; j++) {
      if (B[i] < B[j]) {
           temp = B[i];
               B[i] = B[j];
               B[j] = temp;
           }
        }
    }
for (i = 0; i < n; i++)
    sum += (long long)A[i] * B[i];
  printf("%lld", sum);
   return 0;
}
```

DIVIDE AND CONQUER

1-Number of Zeros in a Given Array

```
#include <stdio.h>
 int main()
{
   int n, a[100], count = 0;
  scanf("%d", &n);
   for (int i = 0; i < n; i++)
    scanf("%d", &a[i]);
   for (int i = 0; i < n; i++)
   if (a[i] == 0)
```

```
   count++;
   printf("%d", count);
   return 0;
}
```

2-Majority Element

```
#include <stdio.h>
 int main()
{
  int n, a[100], t;
  scanf("%d", &n);
  for (int i = 0; i < n; i++)
     scanf("%d", &a[i]);
   for (int i = 0; i < n - 1; i++)
  for (int j = i + 1; j < n; j++)
   if (a[i] > a[j])
 {
   t = a[i];
    a[i] = a[j];
   a[j] = t;
           }
    printf("%d\n", a[n / 2]);
   return 0;
}
```

3-Finding Floor Value

```
#include <stdio.h>
 int main()
{
```

```
  int n, a[100], x, floor = -1;
   scanf("%d", &n);
   for (int i = 0; i < n; i++)
     scanf("%d", &a[i]);
  scanf("%d", &x);
   for (int i = 0; i < n; i++)
    {
  if (a[i] <= x)
    floor = a[i];
    else
   break;
    }
    printf("%d\n", floor);
  return 0;
}
```

4-Two Elements sum to x

```
#include <stdio.h>
 int main()
{
    int n, a[100], x;
    scanf("%d", &n);
   for (int i = 0; i < n; i++)
     scanf("%d", &a[i]);
   scanf("%d", &x);
  for (int i = 0; i < n; i++)
    {
     for (int j = i + 1; j < n; j++)
        {
```

```
        if (a[i] + a[j] == x)
            {
        printf("%d\n%d\n", a[i], a[j]);
            return 0;
            }
        }
    }
    printf("No\n");
 return 0;
}
```

5-Implementation of Quick Sort

```
#include <stdio.h>
 void quickSort(int a[], int n)
{
   for (int i = 0; i < n - 1; i++)
    {
     for (int j = 0; j < n - i - 1; j++)
        {
        if (a[j] > a[j + 1])
     {
     int t = a[j];
   a[j] = a[j + 1];
  a[j + 1] = t;
            }
        }
    }
}
int main()
```

```
{
  int n, a[100];
 scanf("%d", &n);
  for (int i = 0; i < n; i++)
 scanf("%d", &a[i]);
 quickSort(a, n);
 for (int i = 0; i < n; i++)
 printf("%d ", a[i]);
  return 0;
}
```

DYNAMIC PROGRAMMING

1-DP-Playing with Numbers

```
#include <stdio.h>
 int main()
{
  int n;
    scanf("%d", &n);
 long long dp[n + 1];
  dp[0] = 1;
   for (int i = 1; i <= n; i++)
    {
   dp[i] = dp[i - 1];
    if (i >= 3)
    dp[i] += dp[i - 3];
    }
    printf("%lld", dp[n]);
  return 0;
}
```

2-DP-Playing with chessboard

```
#include <stdio.h>
 int main()
{
  int n, a[100][100];
  scanf("%d", &n);
 for (int i = 0; i < n; i++)
   for (int j = 0; j < n; j++)
  scanf("%d", &a[i][j]);
  for (int i = 0; i < n; i++)
    {
    for (int j = 0; j < n; j++)
        {
     if (i > 0 && j == 0)
       a[i][j] += a[i - 1][j];
     else if (i == 0 && j > 0)
      a[i][j] += a[i][j - 1];
    else if (i > 0 && j > 0)
           {
      if (a[i - 1][j] > a[i][j - 1])
        a[i][j] += a[i - 1][j];
     else
 a[i][j] += a[i][j - 1];
           }
        }
    }
    printf("%d", a[n - 1][n - 1]);
   return 0;
}
```

3-DP-Longest Common Subsequence

```
#include <stdio.h>
#include <string.h>
 int main()
{
   char a[1005], b[1005];
   scanf("%s %s", a, b);
   int m = strlen(a), n = strlen(b);
   int dp[1005][1005] = {0};
    for (int i = 1; i <= m; i++)
  for (int j = 1; j <= n; j++)
        {
  if (a[i - 1] == b[j - 1])
  dp[i][j] = dp[i - 1][j - 1] + 1;
 else if (dp[i - 1][j] > dp[i][j - 1])
  dp[i][j] = dp[i - 1][j];
  else
    dp[i][j] = dp[i][j - 1];
        }
    printf("%d", dp[m][n]);
  return 0;
}
```

4-DP-Longest non-decreasing Subsequence

```
#include <stdio.h>
 int main()
{
```

```
  int n, a[100], dp[100], max = 1;
  scanf("%d", &n);
  for (int i = 0; i < n; i++)
  scanf("%d", &a[i]);
     for (int i = 0; i < n; i++)
    {
     dp[i] = 1;
      for (int j = 0; j < i; j++)
        {
    if (a[i] >= a[j] && dp[j] + 1 > dp[i])
            dp[i] = dp[j] + 1;
        }
      if (dp[i] > max)
        max = dp[i];
    }
    printf("%d", max);
   return 0;
}
```

COMPETITIVE PROGRAMMING

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
int main() {
    int n, i, j;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++)
 scanf("%d", &a[i]);
```

```
    for (i = 0; i < n; i++) {
   for (j = i + 1; j < n; j++) {
    if (a[i] == a[j]) {
  printf("%d", a[i]);
  return 0;
           }
       }
    }
    return 0;
}
```

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
 int main() {
    int n, i, x;
   int sum = 0;
   scanf("%d", &n);
   for (i = 0; i < n; i++) {
    scanf("%d", &x);
    sum += x;
    }
    printf("%d", sum - (n * (n - 1)) / 2);
     return 0;
}
```

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
```

```
 int main()
{
  int n, m, a[100], b[100];
 scanf("%d", &n);
  for (int i = 0; i < n; i++)
    scanf("%d", &a[i]);
  scanf("%d", &m);
   for (int i = 0; i < m; i++)
      scanf("%d", &b[i]);
   for (int i = 0; i < n; i++)
    {
    for (int j = 0; j < m; j++)
        {
      if (a[i] == b[j])
            {
                printf("%d ", a[i]);
          break;
            }
        }
    }
    return 0;
}
```

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
 int main() {
    int t, n, m, i, j;
```

```
    scanf("%d", &t);
 while (t--) {
    scanf("%d", &n);
        int a[n];
      for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
     scanf("%d", &m);
   int b[m];
   for (i = 0; i < m; i++)
   scanf("%d", &b[i]);
       i = 0;
      j = 0;
       while (i < n && j < m) {
      if (a[i] == b[j]) {
              printf("%d ", a[i]);
          i++;
   j++;
        }
      else if (a[i] < b[j])
          i++;
      else
   j++;

        }
     printf("\n");
    }
    return 0;
}
```

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
 int main() {
    int n, i, j, k;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++)
   scanf("%d", &a[i]);
   scanf("%d", &k);
  for (i = 0; i < n; i++) {
      for (j = i + 1; j < n; j++) {
      if (a[j] - a[i] == k || a[i] - a[j] == k) {
              printf("1");
           return 0;
            }
       }
  }
    printf("0");
    return 0;
}
```

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
int main()
 {
   int n, k;
   scanf("%d", &n);
    int a[n];
```

```
    for (int i = 0; i < n; i++)
 scanf("%d", &a[i]);
  scanf("%d", &k);
   int i = 0, j = 1;
  while (j < n) {
     int diff = a[j] - a[i];
     if (diff == k) {
   printf("1");
        return 0;
    } else if (diff < k) {
         j++;
     } else {
          i++;              if (i == j)
            j++;
       }
  }
 printf("0");
   return 0;
}
```